Trabalho 2 — Árvores Binárias

Entrega parcial: seções 1 a 3 (árvores binárias não balanceadas)

Instituto Federal do Espírito Santo — Campus Serra
Técnicas de Programação Avançada
Prof. Victorio Albani de Carvalho

Integrantes do grupo

Lucas Ramos Pianissola
Ryan Lima Peçanha
Davi Alvarenga

Repositório do projeto
https://github.com/ryanlimap/trabalho_tpa

Esta é a **entrega parcial** do trabalho e contempla as seções 1, 2 e 3 da especificação. As seções 4 (árvores AVL), 5 (comparação AVL × árvore binária) e 6 (pesquisa sobre as árvores da biblioteca padrão de Java) serão apresentadas na entrega final.

1. Desenvolvimento da biblioteca de árvores e do programa de teste

1.1 Atuação de cada componente do grupo

O quadro abaixo descreve a participação de cada integrante nas etapas 1 e 2 da especificação. Como no trabalho anterior, o trabalho foi dividido por módulo, mas as decisões de projeto e a revisão do código foram discutidas pelos três.

Integrante	Atuação nas etapas 1 e 2
Lucas Ramos Pianissola	Etapa 1 — a biblioteca de árvores: classes `NoArvore` e `ArvoreBinaria` (especializando `ArvoreBinariaBase`), com os métodos `adicionar`, `pesquisar`, `remover`, `quantidadeNos`, `altura`, `caminharEmNivel` e `caminharEmOrdem`; suíte de testes `app.TesteArvore`.
Ryan Lima Peçanha	Etapa 2 — adaptação do aplicativo: o menu inicial passou a oferecer lista não-ordenada, lista ordenada ou árvore binária, mantendo o restante do programa inalterado graças à interface `ICole cao`.
Davi Alvarenga	Infraestrutura das medições: adaptação do `GeradorArquivo` para os três modos de geração, `Benchmark` e `ContagemOperacoes`; execução das rodadas, gráficos e redação do relatório.

1.2 Como as ferramentas de IA foram utilizadas

Seguimos a mesma prática do trabalho anterior, usando uma ferramenta de IA generativa (Claude, da Anthropic) como apoio em frentes bem delimitadas.

a) Aproveitamento do material da disciplina. Fornecemos à ferramenta os slides de árvores da disciplina, os exemplos em C disponibilizados pelo professor (`arvBin.c`) e a classe `ArvoreBinariaBase` do repositório da disciplina. A ferramenta nos ajudou a transpor para Java, com *Generics* e `Comparator`, a lógica de inserção, busca e remoção vista em aula — inclusive a estratégia de remoção de nó com dois filhos por **predecessor** (o maior elemento da subárvore esquerda), que é a mesma usada no exemplo em C.

b) Discussão das decisões de implementação. A principal delas foi **escrever todos os percursos de forma iterativa**, e não recursiva. Os exemplos em C são recursivos, o que é natural em árvores; mas como este trabalho exige inserir centenas de milhares de elementos em uma árvore **degenerada** (altura n−1), a recursão estouraria a pilha da JVM com `StackOverflowError`. Discutimos a alternativa com a ferramenta, implementamos a versão iterativa e confirmamos com um teste que monta uma árvore degenerada de 100.000 nós e a percorre inteira sem estourar a pilha.

c) Infraestrutura de medição e relatório. Os programas geradores de arquivos, as classes de medição e os scripts Python que produzem os gráficos e este PDF foram escritos com apoio da ferramenta, a partir dos dados das nossas execuções. Todos os números apresentados são medições nossas.

d) Revisão de texto. A redação final foi revisada com apoio da ferramenta, para clareza e correção da linguagem.

1.3 Repositório e organização do código

Todo o código está disponível em **https://github.com/ryanlimap/trabalho_tpa**, com um `README.md` descrevendo a organização das pastas e como compilar e executar o projeto.

Pasta / arquivo	Conteúdo
src/colecao/IColecao.java	Interface comum a listas e árvores (do professor).
src/colecao/ListaEncadeada.java	A biblioteca de listas do Trabalho 1, reaproveitada sem alterações.
src/arvorebinaria/ArvoreBinariaBase.java	Classe abstrata fornecida pelo professor, preservada como recebida.
src/arvorebinaria/NoArvore.java	Nó da árvore: valor do tipo T e as referências esquerda e direita.
src/arvorebinaria/ArvoreBinaria.java	**A biblioteca de árvores**: especializa `ArvoreBinariaBase`. Não imprime nada.
src/app/Programa.java	A agenda: agora pergunta qual estrutura usar (lista não-ordenada, lista ordenada ou árvore binária).
src/app/GeradorArquivo.java	Gera os arquivos de entrada em ordem crescente, balanceada ou aleatória.
src/app/Benchmark.java	Uma rodada de medição de tempo; imprime uma linha CSV.
src/app/ContagemOperacoes.java	Conta comparações em vez de medir tempo.
src/app/TesteArvore.java	Suíte de testes automatizados da árvore.
analise/ e dados/	Scripts dos gráficos e deste PDF; CSVs com as medições.

O que mudou no aplicativo. Como a `ArvoreBinariaBase` implementa `IColecao`, a única alteração necessária no programa do Trabalho 1 foi a pergunta inicial e a criação do objeto correspondente. Todo o resto — menu, carga de arquivo, buscas, remoção, alteração e medição de tempos — continua funcionando sem modificação, porque conversa apenas com a interface. Esse é exatamente o ganho de projeto que a interface oferece.

Listagem 1 — a única mudança estrutural no app — src/app/Programa.java, linhas 31-48

```
 31          switch (estrutura) {
 32              case 2 -> {
 33                  colecaoPorNome = new ListaEncadeada<>(new ComparadorContatoPorNome(), true);
 34                  colecaoPorTelefone = new ListaEncadeada<>(new ComparadorContatoPorTelefone(), true
);
 35              }
 36              case 3 -> {
 37                  colecaoPorNome = new ArvoreBinaria<>(new ComparadorContatoPorNome());
 38                  colecaoPorTelefone = new ArvoreBinaria<>(new ComparadorContatoPorTelefone());
 39              }
 40              default -> {
 41                  colecaoPorNome = new ListaEncadeada<>(new ComparadorContatoPorNome(), false);
 42                  colecaoPorTelefone = new ListaEncadeada<>(new ComparadorContatoPorTelefone(), fals
e);
 43              }
 44          }
 45
 46          int opcao;
 47          do {
 48              exibirMenu();
```

2. Análise matemática de complexidade

Nesta seção analisamos os métodos `adicionar`, `pesquisar`, `remover` e `quantidadeNos` da classe `ArvoreBinaria`. A numeração das linhas é a numeração real do arquivo `src/arvorebinaria/ArvoreBinaria.java`.

Premissas. (i) *n* é a quantidade de nós da árvore e *h* a sua altura; (ii) atribuições, comparações, criação de um nó e desvios são operações de custo constante; (iii) a chamada `comparador.compare(...)` é O(1); (iv) analisamos o **pior caso**.

2.1 O ponto central: a topologia manda na complexidade

Diferentemente da lista encadeada, em uma árvore binária o custo dos quatro métodos analisados nesta seção não depende diretamente de *n*, e sim da **altura** da árvore — porque `adicionar`, `pesquisar` e `remover` seguem um **único caminho da raiz até um nó**, e nunca visitam a árvore inteira. (Os caminhamentos e o método `altura`, esses sim, visitam todos os nós e são O(n) em qualquer topologia.) E a altura, para um mesmo *n*, depende da **ordem em que os elementos foram inseridos**. Como visto em aula, vale a relação:

```
                        n - 1  >=  h  >=  piso(log2 n)

h = n - 1           arvore COMPLETAMENTE DEGENERADA: cada no tem um unico
                    filho; a arvore vira uma lista encadeada.
                    Ocorre quando os elementos sao inseridos em ordem
                    crescente (ou decrescente).

h = piso(log2 n)    arvore PERFEITAMENTE BALANCEADA: todos os niveis
                    completos, exceto possivelmente o ultimo (um nivel d
                    comporta ate 2^d nos). Para n = 2^k - 1 todos os niveis
                    ficam cheios; para os demais n, o ultimo nivel fica
                    parcialmente preenchido e a altura ainda e piso(log2 n).
                    Ocorre quando se insere sempre o elemento central do
                    intervalo restante.
```

Portanto, **toda operação que custa O(h) tem dois piores casos a declarar**: O(log n) se a árvore estiver perfeitamente balanceada e O(n) se estiver degenerada. É essa diferença que a seção 3 mede.

2.2 Método adicionar

Listagem 2 — adicionar — src/arvorebinaria/ArvoreBinaria.java, linhas 37-66

```
37      @Override
38      public boolean adicionar(T novoValor) {
39          if (novoValor == null) {
40              return false;
41          }
42          NoArvore<T> novo = new NoArvore<>(novoValor);
43          if (this.raiz == null) {
44              this.raiz = novo;
45              this.quant++;
46              return true;
47          }
48          NoArvore<T> atual = this.raiz;
49          while (true) {
50              if (comparador.compare(atual.getValor(), novoValor) > 0) {
51                  if (atual.getEsq() == null) {
52                      atual.setEsq(novo);
53                      break;
54                  }
55                  atual = atual.getEsq();
56              } else {
57                  if (atual.getDir() == null) {
58                      atual.setDir(novo);
59                      break;
60                  }
61                  atual = atual.getDir();
62              }
63          }
64          this.quant++;
65          return true;
66      }
```

As linhas 38-41 rejeitam `null` e a linha 42 cria o nó: custo constante. As linhas 43-47 tratam a árvore vazia, em que o novo elemento vira a raiz: também constante. O que define a complexidade é o laço das linhas 49-62: partindo da raiz, cada iteração faz **uma comparação** (linha 50) e desce um nível — para a esquerda, se o valor do nó atual for maior que o novo (linhas 51-55), ou para a direita, caso contrário (linhas 56-61). O laço termina quando encontra a posição livre onde o novo nó é pendurado.

Note que elementos de chave **igual** caem no `else` e são inseridos à direita — a mesma convenção do exemplo em C visto em aula.

O laço desce no máximo *h* níveis, então **T(n) = $c_1 \cdot h + c_2$**, ou seja, **O(h)**. Traduzindo para *n*:

Topologia	Altura	Custo de adicionar	Quando ocorre
Perfeitamente balanceada	piso($\log_2$ n)	**O(log n)**	Inserção na ordem “do meio para fora”, como no nosso arquivo balanceado.
Degenerada	n – 1	**O(n)**	Inserção em ordem crescente ou decrescente das chaves — o caso do nosso arquivo em ordem crescente.

Consequência: custo de montar a árvore com n elementos. Inserindo os *n* contatos um a um:

```
Arvore balanceada:  cada insercao custa O(log i) para i elementos ja
                    inseridos. Somando:  log1 + log2 + ... + logn  =
                    log(n!), que e  O(n log n).

Arvore degenerada:  a i-esima insercao percorre os i-1 nos ja inseridos
                    (todos estao em uma unica "espinha"). Somando:
                    0 + 1 + ... + (n-1) = n(n-1)/2  ->  O(n^2).
                    E EXATAMENTE a mesma soma da lista ordenada do
                    Trabalho 1 - porque a estrutura e, de fato, a mesma.
```

2.3 Método pesquisar

Listagem 3 — pesquisar — src/arvorebinaria/ArvoreBinaria.java, linhas 72-83

```
72      @Override
73      public T pesquisar(T valor) {
74          NoArvore<T> atual = this.raiz;
75          while (atual != null) {
76              int cmp = comparador.compare(atual.getValor(), valor);
77              if (cmp == 0) {
78                  return atual.getValor();
79              }
80              atual = (cmp > 0) ? atual.getEsq() : atual.getDir();
81          }
82          return null;
83      }
```

O método segue um único caminho a partir da raiz: a cada nó faz uma comparação (linha 76) e, pelo sinal, decide se devolve o valor (linhas 77-79), desce à esquerda ou desce à direita (linha 80). Se chega a `null`, o elemento não existe (linha 82).

É aqui que está a vantagem potencial da árvore sobre a lista: cada comparação **descarta uma subárvore inteira** — a busca binária que a lista encadeada não permitia, porque agora existe acesso direto aos dois filhos de cada nó. Note, porém, que o tamanho do que se descarta depende da topologia: na árvore **balanceada** cada comparação elimina cerca de metade dos nós restantes, enquanto na **degenerada** a subárvore descartada está sempre vazia e a comparação elimina apenas o próprio nó — exatamente como em uma lista.

Pior caso: O(h), que ocorre quando o elemento procurado está em uma **folha de profundidade máxima** (ou quando não existe e a busca só termina ao chegar a `null` no nível mais profundo). Em números: **O(log n)** na árvore perfeitamente balanceada e **O(n)** na degenerada — nesta última a árvore é uma lista, e a busca volta a ser linear.

Vale registrar o número exato de comparações desse pior caso, porque ele será conferido na seção 3.2.3: buscar uma folha que está no nível *h* exige visitar os *h* nós do caminho mais a própria folha, ou seja, **h + 1 comparações**. Em uma árvore degenerada construída a partir de chaves crescentes, h = n − 1 e a busca faz exatamente **n comparações**.

2.4 Método remover

Listagem 4 — remover — src/arvorebinaria/ArvoreBinaria.java, linhas 89-130

```
 89      @Override
 90      public boolean remover(T valor) {
 91          NoArvore<T> pai = null;
 92          NoArvore<T> atual = this.raiz;
 93          while (atual != null) {
 94              int cmp = comparador.compare(atual.getValor(), valor);
 95              if (cmp == 0) {
 96                  break;
 97              }
 98              pai = atual;
 99              atual = (cmp > 0) ? atual.getEsq() : atual.getDir();
100          }
101          if (atual == null) {
102              return false;
103          }
104          // Caso 3: dois filhos. Seguindo a mesma estratégia vista em aula, o nó
105          // recebe o valor do seu PREDECESSOR (o maior valor da subárvore
106          // esquerda) e passa-se a remover o nó do predecessor, que por ser o
107          // mais à direita da subárvore esquerda tem no máximo um filho.
108          if (atual.getEsq() != null && atual.getDir() != null) {
109              NoArvore<T> paiPredecessor = atual;
110              NoArvore<T> predecessor = atual.getEsq();
111              while (predecessor.getDir() != null) {
112                  paiPredecessor = predecessor;
113                  predecessor = predecessor.getDir();
114              }
115              atual.setValor(predecessor.getValor());
116              pai = paiPredecessor;
117              atual = predecessor;
118          }
119          // Casos 1 e 2: folha (filho == null) ou um único filho.
120          NoArvore<T> filho = (atual.getEsq() != null) ? atual.getEsq() : atual.getDir();
121          if (pai == null) {
122              this.raiz = filho;
123          } else if (pai.getEsq() == atual) {
124              pai.setEsq(filho);
125          } else {
126              pai.setDir(filho);
127          }
128          this.quant--;
129          return true;
130      }
```

A remoção tem duas fases. A **primeira** (linhas 90-103) é uma busca igual à do `pesquisar`, guardando também o nó **pai**, necessário para refazer a ligação: custo **O(h)**.

A **segunda** fase trata os três casos clássicos:

Caso	Linhas	O que acontece	Custo
Nó com dois filhos	108-120	O nó recebe o valor do seu **predecessor** (o maior da subárvore esquerda, encontrado descendo sempre à direita) e passa-se a remover o nó do predecessor, que tem no máximo um filho.	O(h)
Nó folha	122-129	`filho` fica `null` e o pai passa a apontar para `null`.	O(1)
Nó com um filho	122-129	O pai passa a apontar diretamente para o único filho do nó removido.	O(1)

A busca pelo predecessor (laço das linhas 111-114) desce sempre pela direita a partir do filho esquerdo do nó removido. É importante notar que ela **não** acrescenta outros *h* níveis ao custo: a descida até o nó e a descida até o predecessor formam um **único caminho que parte da raiz**, de modo que o total de nós visitados é no máximo **h + 1**. Somando as duas fases, **T(n) = c·h + c'**, ou seja, **O(h)** — novamente **O(log n)** em árvore balanceada e **O(n)** em árvore degenerada.

Quando o pior caso ocorre. Na árvore **balanceada**, ao remover um nó cujo predecessor está no último nível — ou a própria folha de profundidade máxima —, percorrendo os h+1 nós do caminho. Na árvore **degenerada** gerada por chaves crescentes há uma particularidade que vale mencionar na

entrevista: como todo nó tem apenas filho direito, **nenhum nó tem dois filhos** e o caso 3 nunca chega a ser executado; o pior caso é simplesmente remover o último nó da espinha, que custa a varredura completa — O(n).

Vale observar o contraste com a lista do Trabalho 1: lá, a religação em si era O(1) e o custo todo estava em **encontrar** o nó. Aqui vale o mesmo princípio, só que encontrar passou a custar O(h) em vez de O(n) — quando a árvore é balanceada.

2.5 Método quantidadeNos

Listagem 5 — quantidadeNos — src/arvorebinaria/ArvoreBinaria.java, linhas 136-139

```
136      @Override
137      public int quantidadeNos() {
138          return this.quant;
139      }
```

Como na lista, mantemos um contador atualizado na inserção (linhas 45 e 64) e na remoção (linha 128). O método apenas o devolve: **T(n) = 1**, portanto **O(1)**, **independentemente da topologia**. Se o contador não existisse, seria preciso percorrer toda a árvore e o método seria O(n).

2.6 Resumo e comparação com as listas do Trabalho 1

Método	Árvore balanceada	Árvore degenerada	Lista ordenada	Lista não-ordenada
adicionar	O(log n)	O(n)	O(n)	O(1)
pesquisar	O(log n)	O(n)	O(n)	O(n)
remover	O(log n)	O(n)	O(n)	O(n)
quantidadeNos	O(1)	O(1)	O(1)	O(1)
montar n elementos	O(n log n)	$O(n^2)$	$O(n^2)$	O(n)

A leitura da tabela é o resumo do trabalho: **a árvore binária só é melhor que a lista enquanto a sua topologia for favorável**. Perfeitamente balanceada, ela transforma O(n) em O(log n) nas três operações principais. Degenerada, ela **é** uma lista encadeada — com o agravante de gastar um ponteiro a mais por nó, já que o campo da esquerda fica sempre nulo.

É por isso que existem as árvores balanceadas (AVL), objeto das próximas seções deste trabalho: elas garantem h = O(log n) **independentemente da ordem de inserção**, eliminando o pior caso que medimos aqui.

3. Análise empírica de complexidade

3.1 Implementação: geração dos arquivos e definição do pior caso

Como geramos os arquivos de entrada. Adaptamos o gerador do Trabalho 1 (classe `app.GeradorArquivo`) para produzir os contatos em três ordens diferentes, controlando a topologia da árvore resultante. Cada contato tem um identificador *i*; o telefone é a **concatenação** do prefixo `"27"` com o número `900000000 + i` e o nome é `"Contato "` seguido de *i* com zeros à esquerda. Para *i* = 5, por exemplo, o registro gerado é `Contato 0000005;27900000005` — o nome é derivado do mesmo identificador **para que a ordem alfabética dos nomes seja idêntica à ordem numérica dos telefones**, de modo que as duas árvores (a indexada por nome e a indexada por telefone) tenham a mesma topologia controlada.

Os dois modos de geração usados nos testes

```
CRESCENTE  (arquivo cres_N.txt)  ->  arvore DEGENERADA
   Emite os identificadores 0, 1, 2, 3, ... em ordem.
   Cada novo elemento e maior que todos os ja inseridos, entao desce
   sempre para a direita e vira filho direito do ultimo. A arvore nao
   tem nenhum filho a esquerda: altura = n - 1.

BALANCEADO (arquivo bal_N.txt)   ->  arvore PERFEITAMENTE BALANCEADA
   Emite o elemento do MEIO do intervalo [0, n-1]; depois o meio de cada
   metade; depois o meio de cada quarto, e assim por diante (percurso em
   largura sobre os intervalos).
   Exemplo com n = 15:  7 | 3 11 | 1 5 9 13 | 0 2 4 6 8 10 12 14
   Inserindo nessa ordem, cada elemento cai exatamente no lugar que
   completa o nivel: altura = piso(log2 n).
```

Como definimos o pior caso de busca. A especificação pede uma busca pela folha mais distante da raiz, considerando a indexação por telefone. Em vez de deduzir qual elemento seria esse em cada arquivo, a própria biblioteca responde: implementamos na `ArvoreBinaria` o método `valorDaFolhaMaisProfunda()`, que faz um percurso em largura e devolve o valor do **último nó visitado** — que está necessariamente no nível mais profundo, e portanto é uma folha de profundidade máxima. Essa é, por definição, a busca com sucesso mais cara da árvore.

Listagem 6 — identificação do pior caso de busca — src/arvorebinaria/ArvoreBinaria.java, linhas 236-254

```
236     public T valorDaFolhaMaisProfunda() {
237         if (this.raiz == null) {
238             return null;
239         }
240         NoArvore<T> ultimo = this.raiz;
241         Deque<NoArvore<T>> fila = new ArrayDeque<>();
242         fila.add(this.raiz);
243         while (!fila.isEmpty()) {
244             NoArvore<T> no = fila.poll();
245             ultimo = no;
246             if (no.getEsq() != null) {
247                 fila.add(no.getEsq());
248             }
249             if (no.getDir() != null) {
250                 fila.add(no.getDir());
251             }
252         }
253         return ultimo.getValor();
254     }
```

O método é chamado sobre a **árvore indexada por telefone**, como a especificação determina, e o contato devolvido por ele é o **mesmo** usado nas três medições seguintes: a busca por telefone, a busca por nome e a remoção por telefone.

Sobre usar uma classe de medição separada. As medições são feitas pela classe `app.Benchmark`, e não digitando as opções no menu do `app.Programa`. Ela executa exatamente a mesma sequência de operações que o menu executaria, sobre **as mesmas bibliotecas** e através da **mesma interface** `IColecao` — a única diferença é que ela dispensa a digitação e imprime os tempos já em formato CSV, o que torna viável repetir dezenas de execuções de forma automatizada e reprodutível.

Metodologia das medições. Para cada arquivo, a classe `app.Benchmark` (i) lê o arquivo e monta as duas árvores, cronometrando o tempo total; (ii) identifica a folha mais profunda; (iii) pesquisa esse contato pelo telefone; (iv) pesquisa o mesmo contato pelo nome; (v) remove-o pelo telefone. Os tempos são medidos com `System.nanoTime()` e cada medição roda em uma **JVM nova**. O tempo de **montagem** é a mediana de 3 execuções completas. Já as três **operações** (busca por telefone, busca por nome e remoção) são repetidas **5 vezes dentro de cada execução**, e reportamos a mediana dessas repetições — o contato removido é reinserido após cada medição, de modo que todas partem do mesmo estado. Isso foi necessário porque essas operações duram de microssegundos a poucos milissegundos, faixa em que uma medida isolada é dominada por ruído do sistema operacional e do coletor de lixo. Ao todo, cada ponto das tabelas de operação resume 15 medições.

Além do tempo, a classe `app.ContagemOperacoes` conta o **número de comparações** executadas. Como no Trabalho 1, essa medida independe de máquina, JIT e cache, e é a evidência mais limpa da ordem de crescimento.

Ambiente de execução: OpenJDK 21 (64 bits) sobre Linux x86-64, com memória máxima de heap de 3 GB para a JVM. Todas as medições desta seção foram coletadas na mesma máquina, sem outros programas concorrendo pela CPU.

Tamanhos usados: 25.000, 50.000, 100.000, 200.000 contatos, os mesmos do Trabalho 1, em dois arquivos cada (crescente e balanceado).

3.2 Análise de complexidade

Altura das árvores geradas. Antes dos tempos, a medida que explica todo o resto:

n	Altura — balanceada	piso($\log_2$ n)	Altura — degenerada	n − 1
25.000	14	14	24.999	24.999
50.000	15	15	49.999	49.999
100.000	16	16	99.999	99.999
200.000	17	17	199.999	199.999

Os geradores fizeram exatamente o que deveriam: a árvore do arquivo balanceado atingiu a **altura mínima teórica** piso($\log_2$ n) em todos os tamanhos, e a do arquivo crescente atingiu a **altura máxima** n−1. Com 200.000 contatos, uma árvore tem altura 17 e a outra 199.999: é essa diferença de quatro ordens de grandeza que aparece em todos os gráficos a seguir.

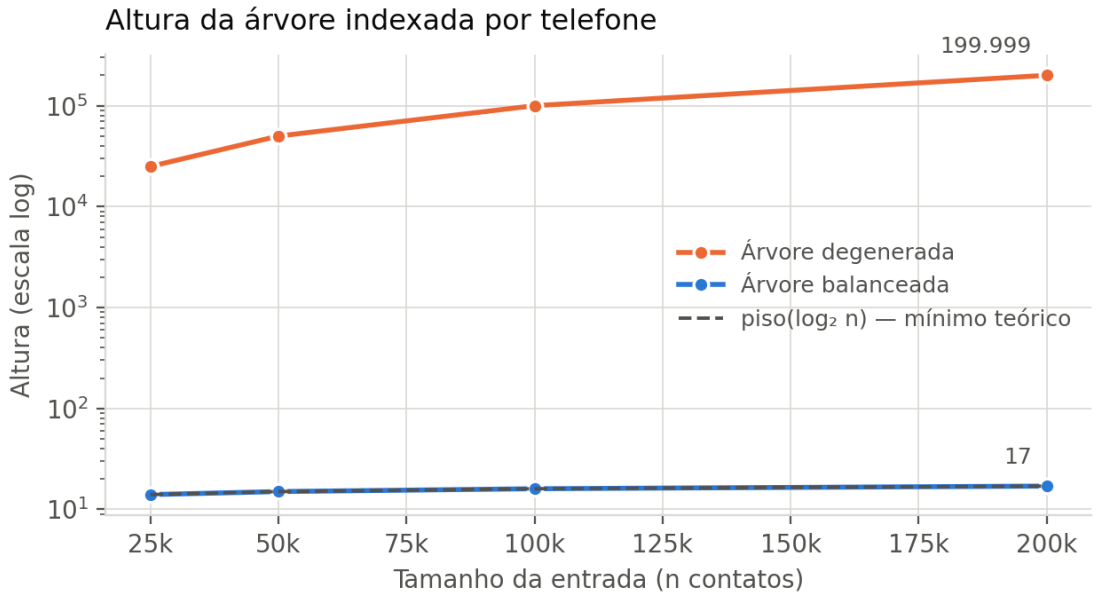


Figura 1 — Altura das árvores (escala logarítmica). A curva da árvore balanceada coincide com o mínimo teórico piso($\log_2$ n).

3.2.1 Tempo de leitura do arquivo e montagem das árvores

n	Montagem (ms) árvore balanceada	razão	Montagem (ms) árvore degenerada	razão
25.000	68,82	—	8.461,14	—
50.000	100,37	1,46×	32.658,90	3,86×
100.000	171,01	1,70×	130.083,80	3,98×
200.000	289,34	1,69×	505.560,13	3,89×

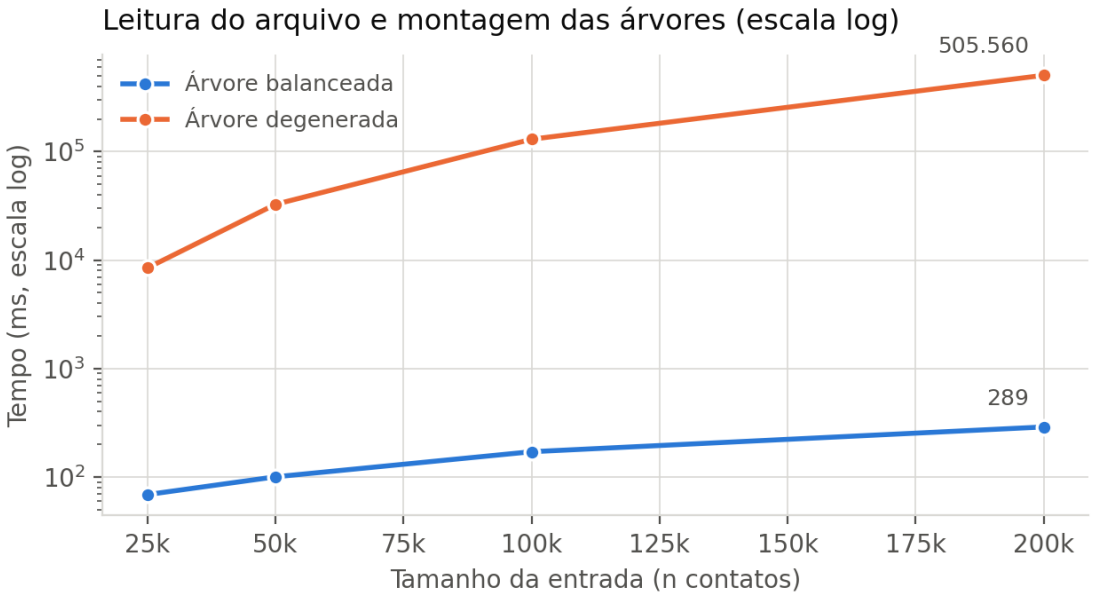


Figura 2 — Montagem das árvores (escala logarítmica).

Interpretação. Na árvore **balanceada**, dobrar *n* multiplica o tempo por valores entre 1,46×, 1,70× e 1,69× — compatível com o O(n log n) previsto na seção 2.2. O fator teórico ao dobrar *n* é 2·log(2n)/log(n), que para esses tamanhos vale cerca de 2,1; as razões medidas ficam abaixo disso porque parte do tempo não cresce com *n* da mesma forma: a compilação JIT dos métodos acontece nas primeiras milhares de inserções e a abertura do arquivo tem custo próprio. Note que a inicialização da JVM **não** entra nessa conta — o cronômetro `System.nanoTime()` só começa dentro do programa, depois que a máquina virtual já subiu.

Na árvore **degenerada**, as razões são 3,86×, 3,98× e 3,89× a cada duplicação de *n* — a assinatura de **O(n^2)**, exatamente o previsto na seção 2.2. Com 200.000 contatos a montagem degenerada levou **8,4 minutos**, contra **289 ms** da balanceada: cerca de **1.747 vezes mais**, com o **mesmo número de elementos e o mesmo código**. A única diferença entre as duas execuções é a ordem em que os dados chegaram.

Vale notar que essas razões medidas no relógio ficam ligeiramente **abaixo** de 4, e não acima — ou seja, diferentemente do que observamos com as listas do Trabalho 1, aqui o custo por operação não se deteriorou com o tamanho. Uma explicação plausível é que a inserção degenerada percorre sempre a mesma "espinha" de nós, acessada repetidamente a cada nova inserção, o que é mais amigável ao cache do que o padrão de acesso da lista ordenada. De qualquer forma, a medida que decide a questão é a **contagem de comparações** da seção 3.2.3, que cresce exatamente 4,00× a cada duplicação.

3.2.2 Tempo de pesquisa e de remoção no pior caso

n	Pesquisa por telefone (ms) árvore balanceada	razão	Pesquisa por telefone (ms) árvore degenerada	razão
25.000	0,0008	—	1,0650	—
50.000	0,0009	1,12×	0,7336	0,69×
100.000	0,0009	1,00×	1,3362	1,82×
200.000	0,0011	1,22×	2,8251	2,11×

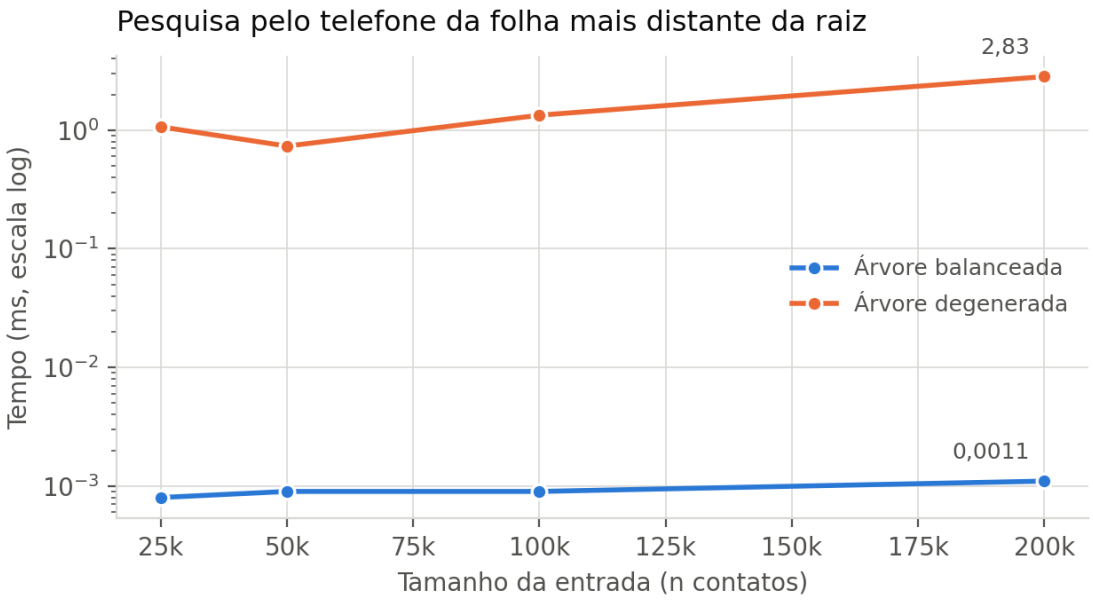


Figura 3 — Pesquisa pelo telefone da folha mais distante da raiz.

n	Pesquisa por nome (ms) árvore balanceada	razão	Pesquisa por nome (ms) árvore degenerada	razão
25.000	0,0009	—	0,3383	—
50.000	0,0009	1,00×	0,7670	2,27×
100.000	0,0010	1,11×	1,4562	1,90×
200.000	0,0011	1,10×	2,9646	2,04×

n	Remoção (ms) árvore balanceada	razão	Remoção (ms) árvore degenerada	razão
25.000	0,0010	—	1,0055	—
50.000	0,0010	1,00×	0,9612	0,96×
100.000	0,0011	1,10×	1,3023	1,35×
200.000	0,0011	1,00×	2,7955	2,15×

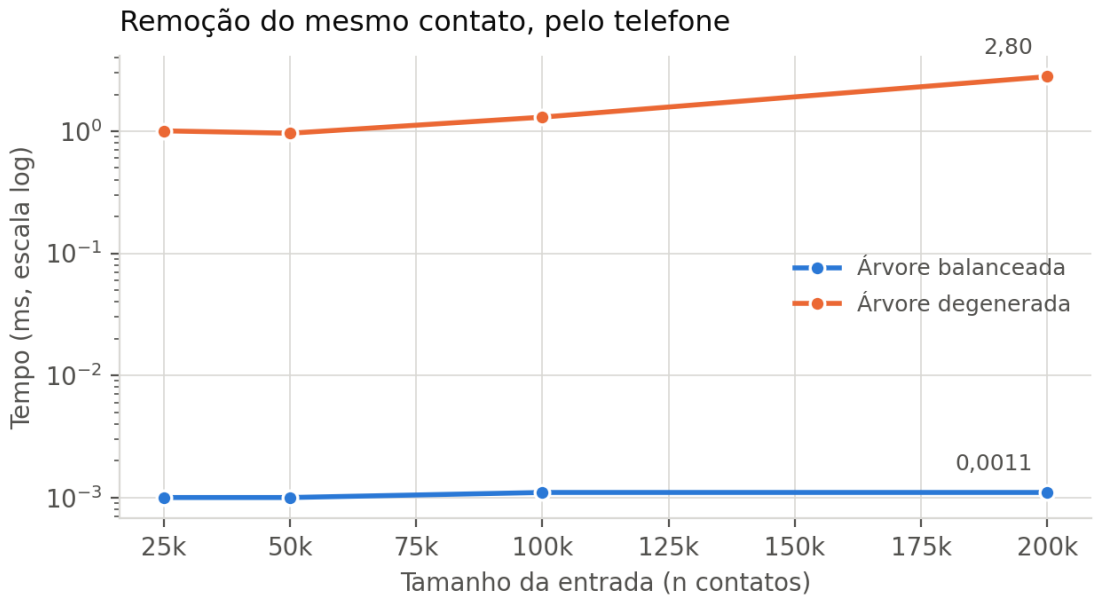


Figura 4 — Remoção do mesmo contato, pesquisado pelo telefone.

Interpretação. As três tabelas contam a mesma história, e é a prevista na seção 2. Na árvore **balanceada**, as duas buscas e a remoção custam cerca de **um microssegundo** (0,001 ms) e praticamente não crescem: entre 25.000 e 200.000 contatos — oito vezes mais dados — a altura sobe apenas de 14 para 17, e é a altura que determina o custo. É o comportamento **O(log n)**: multiplicar *n* por 8 acrescenta 3 níveis à árvore, e portanto 3 comparações à busca.

Na árvore **degenerada**, as mesmas operações custam **milissegundos** — duas a três ordens de grandeza a mais — e crescem com *n*, o comportamento **O(n)**, porque percorrer a árvore degenerada é percorrer uma lista. A confirmação exata vem da contagem de operações (seção 3.2.3): a busca de pior caso faz **exatamente n comparações** na árvore degenerada e **h + 1** na balanceada.

Por que as razões de tempo oscilam. Em termos absolutos, essas operações duram de microssegundos (balanceada) a poucos milissegundos (degenerada) — faixa em que o relógio sofre interferência do escalonador do sistema operacional, do coletor de lixo e do estado dos caches. Mesmo com a mediana de 15 medições por ponto, algumas razões ficam abaixo de 1, ou seja, o tempo medido **caiu** ao dobrar *n*. Isso não contradiz a análise: significa apenas que, nessa escala, a variação do ruído é maior que a diferença real entre dois pontos vizinhos. A tendência entre os extremos (25.000 e 200.000) e a contagem de comparações são as evidências confiáveis, e ambas confirmam O(log n) e O(n).

Dois pontos específicos que merecem comentário. Primeiro, a **remoção na árvore degenerada** é consistentemente próxima do tempo da busca pelo mesmo elemento. Isso é esperado: a remoção é uma busca seguida de um número fixo de religações de ponteiros, e nesta topologia o nó removido é o último da espinha — uma folha, portanto o caso mais barato da segunda fase, sem predecessor a procurar. Segundo, nos tamanhos menores a **busca por nome** chega a ser mais rápida que a busca por telefone na degenerada, embora as duas árvores tenham a mesma topologia e a mesma altura. O motivo é que a busca por nome roda logo depois da busca por telefone dentro de cada repetição, aproveitando o aquecimento do JIT; nos tamanhos maiores, em que o trabalho real domina o ruído, as duas praticamente se igualam, como deveria ser. É um ganho de fator constante, não de ordem.

3.2.3 Contagem de operações

A tabela mostra quantas **comparações entre contatos** foram efetivamente executadas para montar as duas árvores (soma das duas) e para fazer a busca de pior caso.

n	Montagem — balanceada (soma das 2 árvores)	razão	Montagem — degenerada (soma das 2 árvores)	razão	Busca — balanceada (1 árvore)	Busca — degenerada (1 árvore)
25.000	634.496	—	624.975.000	—	15	25.000
50.000	1.368.962	2,16×	2.499.950.000	4,00×	16	50.000
100.000	2.937.892	2,15×	9.999.900.000	4,00×	17	100.000
200.000	6.275.750	2,14×	39.999.800.000	4,00×	18	200.000

Os números confirmam a análise matemática com precisão. Na árvore **degenerada** a contagem é multiplicada por **4,00** a cada duplicação de *n* — a assinatura de n^2 —, e mais do que isso: ela bate **exatamente** com a fórmula deduzida na seção 2.2. Lá chegamos a n(n–1)/2 comparações por árvore; como montamos duas árvores, o total previsto é n(n–1). Para n = 25.000 isso dá 624.975.000, e foi exatamente esse o valor medido — até a última unidade. O mesmo vale para os demais tamanhos.

Na árvore **balanceada** a contagem cresce por pouco mais de **2** a cada duplicação: é o comportamento n·log n, em que dobrar *n* dobra o número de inserções e ainda acrescenta um nível a cada uma. E a busca de pior caso confirma a previsão da seção 2.3 nos dois extremos: na balanceada ela faz **h + 1** comparações (15, 16, 17 e 18 para alturas 14, 15, 16 e 17), e na degenerada faz **exatamente n** comparações — o pior caso linear acontecendo literalmente.

3.2.4 Comparação com as listas do Trabalho 1

Para que a comparação fosse justa, rodamos também as listas encadeadas do Trabalho 1 **na mesma máquina e com os mesmos arquivos**. Tempos de montagem, em milissegundos:

n	Árvore balanceada	Árvore degenerada	Lista não-ordenada	Lista ordenada
25.000	68,8	8.461,1	50,8	4.853,7
50.000	100,4	32.658,9	68,8	20.067,0
100.000	171,0	130.083,8	114,2	87.169,3
200.000	289,3	505.560,1	166,0	355.085,1

Tempos de pesquisa no pior caso, em milissegundos:

n	Árvore balanceada	Árvore degenerada	Lista não-ordenada	Lista ordenada
25.000	0,0008	1,0650	0,7713	0,7477
50.000	0,0009	0,7336	1,3898	0,8811
100.000	0,0009	1,3362	4,3927	1,6574
200.000	0,0011	2,8251	2,4427	3,0587

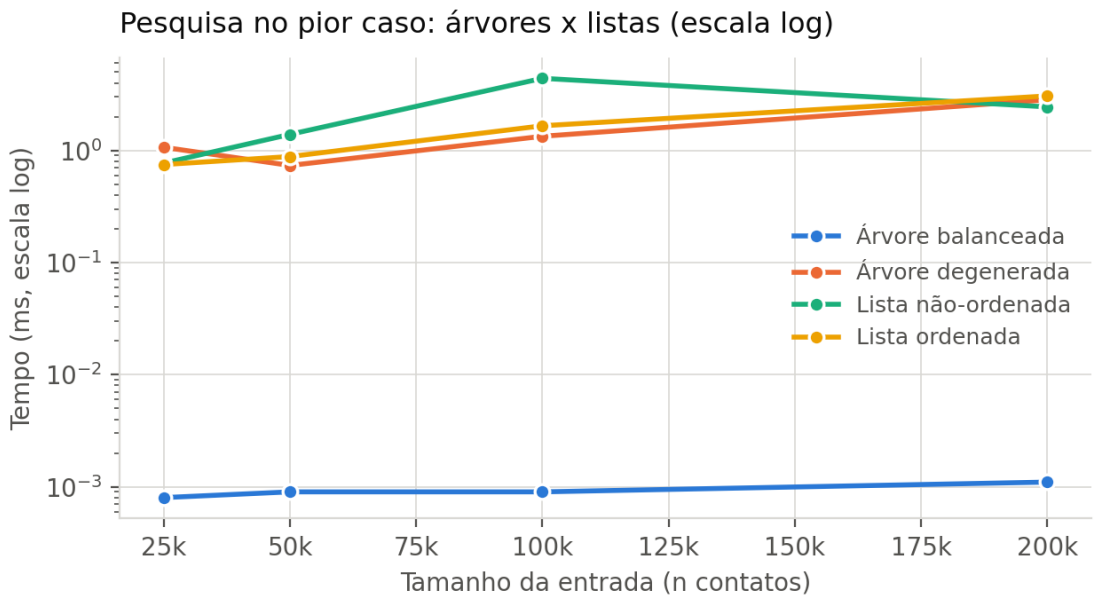


Figura 5 — Pesquisa no pior caso: as quatro estruturas (escala logarítmica).

O que a comparação mostra. Para a **busca**, a árvore balanceada é disparadamente a melhor estrutura: ela resolve em microssegundos o que as listas resolvem em milissegundos, e a vantagem **aumenta** com *n*, porque O(log n) e O(n) divergem. A árvore degenerada, por sua vez, tem desempenho equivalente ao da lista — o que era de se esperar, já que ela **é** uma lista encadeada com um ponteiro desperdiçado por nó.

Para a **montagem**, a lista não-ordenada continua imbatível (O(n), com inserção O(1) no fim graças à referência `ult`), mas ela paga esse preço em toda busca posterior. A árvore balanceada monta em O(n log n) — mais caro que a lista não-ordenada e muito mais barato que a lista ordenada ou que a árvore degenerada, ambas quadráticas.

Duas observações sobre a leitura dessas tabelas. A primeira é que as medidas da **lista não-ordenada** na busca também oscilam entre tamanhos vizinhos, pelo mesmo motivo já discutido na seção 3.2.2 — são poucos milissegundos, e o ruído de medição é da mesma ordem da diferença entre dois pontos. A tendência entre os extremos, no entanto, é claramente linear, como manda o O(n) da busca pelo último nó. A segunda é que a **lista ordenada** foi medida com o arquivo balanceado, e não com o crescente: montar uma lista ordenada a partir de chaves em ordem crescente é o pior caso absoluto dessa estrutura (cada inserção percorre a lista inteira), e o tempo seria ainda maior que o já medido — o que não acrescentaria nada à comparação, já que o ponto é o mesmo da árvore degenerada.

A conclusão prática é que **a escolha da estrutura depende do perfil de uso**: se os dados são inseridos uma vez e consultados muitas, a árvore balanceada compensa largamente o custo de montagem. Mas essa conclusão tem uma condição embutida — que a árvore **esteja** balanceada —, e nada no código garante isso: basta o arquivo chegar ordenado para cairmos no pior caso.

3.3 Conclusão parcial

Os dados coletados **atendem ao esperado** pela análise matemática da seção 2:

• as alturas medidas bateram exatamente com os limites teóricos — piso($\log_2$ n) na árvore gerada pelo arquivo balanceado e n–1 na gerada pelo arquivo crescente;
• a **montagem** da árvore balanceada cresceu como O(n log n) e a da degenerada como O(n^2), com

a contagem de comparações quadruplicando a cada duplicação de *n*;
• **pesquisa** e **remoção** no pior caso ficaram em microssegundos na árvore balanceada e em milissegundos na degenerada — diferença de duas a três ordens de grandeza —, e a contagem de comparações confirmou O(log n) e O(n) com exatidão (h+1 e n comparações, respectivamente), ainda que os tempos de relógio oscilem entre tamanhos vizinhos por serem medidas muito curtas;
• `quantidadeNos` é O(1) nas duas topologias e seu tempo é indistinguível de zero, por isso não foi incluído nos gráficos.

A lição central desta etapa é que, em uma árvore binária de busca **sem balanceamento**, a complexidade não é uma propriedade do algoritmo isoladamente: ela depende da **ordem em que os dados chegam**. O mesmo código, com os mesmos 200.000 contatos, foi de 17 níveis para 199.999 e de milissegundos para minutos. Em um sistema real, dados costumam chegar ordenados (por data de cadastro, por matrícula, por código sequencial), que é justamente o pior caso.

É exatamente esse problema que as **árvores AVL** resolvem, reequilibrando a árvore após cada inserção e remoção para garantir h = O(log n) em qualquer ordem de entrada. A implementação e a avaliação da AVL são o objeto das seções 4 e 5, na entrega final deste trabalho.